# Final Project: Track A

## 1 Project Overview

Machine learning is increasingly used in biomedical research to predict disease outcomes, identify biological patterns, and investigate relationships between biological measurements and phenotypes. Different machine learning algorithms make different assumptions about relationships between predictors and outcomes, and their performance can vary depending on dataset characteristics.

In this project, you will apply and compare five methods covered in this course: linear regression, logistic regression, decision trees, random forests, and multilayer perceptrons (MLPs). You will investigate two biomedical prediction problems: one regression task and one binary classification task.

The primary goal is **not simply to obtain the highest predictive performance**, but to understand how the algorithms work in practice, how complexity affects generalization, and how to evaluate and interpret machine learning models correctly. You are expected to design reproducible experiments, justify methodological decisions, interpret results, and draw scientifically supported conclusions.

## 2 Learning Objectives

Upon completing this project, you should be able to:

- Formulate regression and classification problems using biomedical data.
- Perform exploratory data analysis and identify important dataset characteristics.
- Implement appropriate preprocessing without introducing data leakage.
- Train linear and nonlinear machine learning models.
- Use cross-validation to select models and tune hyperparameters.
- Evaluate performance on an independent test set.
- Investigate overfitting, underfitting, and the effects of model complexity.
- Compare predictive performance and interpretability across algorithms.
- Explain biological relevance and limitations of model findings.
- Communicate results in a reproducible scientific report.

## 3 Datasets

You will use two publicly available biomedical datasets distributed with scikit-learn.

### 3.1 Dataset A: Diabetes Progression (Regression)

**Python function:** `sklearn.datasets.load_diabetes`

- 442 samples and 10 numerical predictors representing baseline clinical measurements.
- A continuous response representing a quantitative measure of diabetes disease progression one year after baseline.

**Research question:** How accurately can we predict diabetes disease progression from baseline clinical measurements?

Compare linear regression, decision tree regression, random forest regression, and MLP regression.

**Documentation:** https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_diabetes.html

### 3.2 Dataset B: Wisconsin Diagnostic Breast Cancer (Classification)

**Python function:** `sklearn.datasets.load_breast_cancer`

- 569 samples and 30 numerical predictors derived from cell nucleus measurements.
- A binary response indicating malignant or benign tumors.

**Research question:** How accurately can we classify breast tumor samples as malignant or benign using quantitative cell nucleus measurements?

Compare logistic regression, decision tree classification, random forest classification, and MLP classification.

**Documentation:** https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html

For this project, define **malignant as the positive class (1)** and benign as the negative class (0). This requires recoding the original scikit-learn target labels. Both datasets are intended for educational model development; results should not be interpreted as validated clinical prediction systems.

## 4 Project Tasks

### 4.1 Part 1. Exploratory Data Analysis and Preprocessing (15 points)

#### 4.1.1 Dataset exploration

For both datasets, report the number of samples and predictors, meaning of the response and predictors, presence or absence of missing values, summary statistics, and distribution of the response or class labels. Produce **at least two informative visualizations for each dataset**, such as histograms, boxplots, scatterplots, or correlation heatmaps.

#### 4.1.2 Data splitting

Split each dataset into an **80% development set** and a **20% independent test set**. Use `random_state=42`. Stratify the classification split to approximately preserve class proportions. Hold the independent test set aside until final model evaluation.

#### 4.1.3 Preprocessing

Develop appropriate preprocessing pipelines for each algorithm. Consider missing values, standardization, and differences between linear/MLP and tree-based methods. Use scikit-learn `Pipeline` objects so learned preprocessing parameters are estimated only from the appropriate training folds.

**Questions to answer:**

1. Why is standardization important for logistic regression and MLPs?
2. Why are decision trees and random forests generally less sensitive to feature scaling?

3. What is data leakage, and how does a pipeline help prevent it?

## 4.2 Part 2. Regression Modeling (20 points)

Using Dataset A, implement linear regression, decision tree regression, random forest regression, and MLP regression.

### 4.2.1 Baseline model

Train ordinary linear regression as the primary baseline. Report cross-validation performance and examine fitted coefficients.

### 4.2.2 Nonlinear models

Train the remaining three models. Investigate the following hyperparameters using 5-fold cross-validation on the development set. A small, justified search grid is sufficient.

| Algorithm | Hyperparameters |
|---|---|
| Decision tree | `max_depth`, `min_samples_leaf` |
| Random forest | `n_estimators`, `max_depth` |
| MLP | `hidden_layer_sizes`, `alpha` |

### 4.2.3 Performance evaluation

For each algorithm, report the mean and standard deviation across five cross-validation folds of mean squared error (MSE), root mean squared error (RMSE), mean absolute error (MAE), and $R^2$. Use **RMSE as the primary regression selection metric**. After model selection, refit each selected algorithm configuration on the entire development set and evaluate it once on the independent test set.

### 4.2.4 Required outputs

- A table comparing cross-validation performance for all four algorithms.
- A table comparing independent test performance.
- An observed-versus-predicted scatterplot for the best model selected by cross-validation.
- A residual plot for the same model.

**Questions to answer:**

1. Which regression algorithm performs best according to cross-validation RMSE?
2. Does the best nonlinear model outperform linear regression?
3. Are there substantial differences between cross-validation and independent test performance?
4. What might explain the observed differences?

## 4.3 Part 3. Classification Modeling (20 points)

Using Dataset B, implement logistic regression, decision tree classification, random forest classification, and MLP classification.

#### 4.3.1 Baseline model

Use logistic regression as the primary baseline and investigate the effect of its regularization parameter `C`.

#### 4.3.2 Nonlinear models

Use stratified 5-fold cross-validation on the development set to investigate:

| Algorithm | Hyperparameters |
|---|---|
| Decision tree | `max_depth`, `min_samples_leaf` |
| Random forest | `n_estimators`, `max_depth` |
| MLP | `hidden_layer_sizes`, `alpha` |

#### 4.3.3 Performance evaluation

Evaluate accuracy, precision, recall (sensitivity), F1-score, and ROC AUC. Use **ROC AUC as the primary classification selection metric**. Compute ROC AUC from predicted probabilities or continuous decision scores, not hard class predictions. Use a probability threshold of 0.5 for class predictions unless another threshold is justified using development data only. Report cross-validation means and standard deviations. Refit selected configurations and evaluate them on the independent test set.

#### 4.3.4 Required outputs

- A table comparing cross-validation performance for all four classifiers.
- A table comparing independent test performance.
- A confusion matrix for the best classifier selected by cross-validation.
- An ROC curve comparing all four classifiers using independent test predictions.

**Questions to answer:**

1. Which algorithm achieves the highest cross-validation ROC AUC?
2. Does the MLP outperform logistic regression?
3. Which classifier has the highest sensitivity for detecting malignant tumors?
4. Why might sensitivity be particularly important for this biomedical application?
5. Can a classifier have high accuracy but relatively poor sensitivity? Explain.

### 4.4 Part 4. Hyperparameter Tuning and Overfitting (20 points)

Investigate how model complexity affects training performance and generalization.

#### 4.4.1 Experiment A: Decision Tree Complexity

Using Dataset A (regression):

1. Train decision tree regressors with `max_depth` values 1, 2, 3, 4, 5, 7, 10, and `None`.
2. Keep other hyperparameters fixed.

3. For each setting, calculate training RMSE and mean 5-fold cross-validation RMSE using only development data.
4. Plot training and cross-validation RMSE against tree depth. Treat `None` as a categorical setting, not a numeric depth.

**Questions:** How does training error change as depth increases? When does complexity stop improving cross-validation performance? Which settings suggest overfitting? How does limiting depth act as regularization?

#### 4.4.2 Experiment B: MLP Regularization

Using Dataset B (classification):

1. Fix the MLP architecture to one hidden layer with 50 neurons.
2. Investigate `alpha` values 0.0001, 0.001, 0.01, 0.1, and 1.0.
3. Keep other hyperparameters fixed.
4. Calculate training and cross-validation ROC AUC for each value.
5. Plot training and cross-validation ROC AUC against `alpha` on a logarithmic horizontal axis.

Use standardized features and a sufficiently large `max_iter`; check convergence warnings.

**Questions:** How does regularization affect training performance? Does stronger regularization always improve cross-validation performance? Which settings suggest overfitting or underfitting? How does `alpha` differ from tree depth?

For both experiments, use only development data; do not use the independent test set to investigate complexity.

### 4.5 Part 5. Model Comparison and Interpretation (15 points)

#### 4.5.1 Linear versus nonlinear models

Compare linear regression against nonlinear regressors, and logistic regression against nonlinear classifiers. Discuss whether complexity consistently improves performance.

#### 4.5.2 Feature interpretation

For each dataset:

1. Examine coefficients of the linear or logistic regression model.
2. Calculate permutation importance for the selected final model using development data and an appropriate evaluation procedure.
3. Identify three predictors that appear important for prediction.

When comparing coefficients, consider feature standardization and correlated predictors.

#### 4.5.3 Biological interpretation

For each dataset, discuss the biomedical meaning of at least two important predictors. Use dataset documentation and, where appropriate, scientific literature. Explain whether findings are biologically plausible. Predictive feature importance does not establish causation.

#### 4.5.4 Final discussion

1. Which algorithms would you recommend for the two biomedical prediction tasks?
2. Are the most complex algorithms necessarily best?
3. How do sample size, predictor count, regularization, and model flexibility influence performance?
4. What are the limitations of the datasets and modeling procedures?
5. What additional validation would be necessary before using these models in biomedical research or clinical applications?

# 5 Required Methodological Practices

**Data splitting**
Use an 80/20 development/test split. Test data must not influence preprocessing decisions, feature selection, tuning, or model selection.

**Cross-validation**
Use shuffled 5-fold `KFold` for regression and `StratifiedKFold` for classification, with `random_state=42`.

**Pipelines**
Use scikit-learn pipelines for all data-dependent preprocessing.

**Hyperparameter tuning**
Use `GridSearchCV` or another appropriate cross-validation search. Report the search space and selection metric.

**Reproducibility**
Set random seeds where applicable and document software versions and model settings.

**Convergence**
Check MLP convergence warnings. If necessary, adjust optimization settings using development data and document changes.

**Model evaluation**
Select configurations using development-set cross-validation. Refit selected configurations on all development data and evaluate each once on the independent test set. Choose the overall preferred algorithm using development results, not test-set rankings.

**Scientific interpretation**
Consider both predictive performance and biological relevance; avoid unsupported causal claims.

# 6 Deliverables

## 6.1 Deliverable 1: Jupyter Notebooks

Submit `regression.ipynb` and `classification.ipynb`. Include executable code, explanatory Markdown, figures, and interpretations. Each notebook must run from beginning to end.

## 6.2 Deliverable 2: Scientific Report

Submit a PDF report of approximately 5–7 pages, excluding references and appendices. Include:

- **Introduction:** Biological prediction problems and objectives.

- **Data and Methods:** Datasets, preprocessing, models, cross-validation, tuning, and metrics.
- **Results:** Comparison tables, figures, and overfitting experiments.
- **Discussion:** Performance differences, complexity, important predictors, biological relevance, and limitations.
- **Conclusions:** Main findings and model recommendations.
- **References:** Dataset sources and scientific literature.

### 6.3 Deliverable 3: Reproducibility Information

Include a `README.md` listing Python version, required packages, notebook execution instructions, and any additional files. Do not submit unnecessary intermediate files or large generated outputs.

### 6.4 Deliverable 4: Presentation

Prepare and deliver a **5-minute presentation** summarizing your project. Use approximately **4–5 slides** and address:

1. **Research objectives and datasets:** Introduce the regression and classification problems.
2. **Methods:** Summarize the models, preprocessing, and evaluation strategy.
3. **Key results:** Compare model performance and highlight a finding about model complexity or overfitting.
4. **Discussion and conclusions:** Explain the most important findings, biological relevance, and limitations.

Focus on the main findings rather than presenting every experiment. Present during the final exam time and submit your presentation slides with the other deliverables.

# 7 Grading Rubric

| Component | Points |
|---|---|
| **Part 1. EDA and preprocessing** | **15** |
| Dataset understanding and visualizations | 6 |
| Data splitting and preprocessing pipelines | 6 |
| Explanation of preprocessing and leakage | 3 |
| **Part 2. Regression modeling** | **20** |
| Correct implementation of four models | 8 |
| Cross-validation and hyperparameter tuning | 5 |
| Evaluation metrics, figures, and interpretation | 7 |
| **Part 3. Classification modeling** | **20** |
| Correct implementation of four classifiers | 8 |
| Cross-validation and hyperparameter tuning | 5 |
| Evaluation metrics, figures, and interpretation | 7 |
| **Part 4. Model complexity and overfitting** | **20** |
| Decision tree experiment | 8 |
| MLP regularization experiment | 8 |
| Interpretation of generalization behavior | 4 |
| **Part 5. Model comparison and interpretation** | **15** |
| Linear versus nonlinear comparison | 4 |
| Feature interpretation | 5 |
| Biological discussion, limitations, recommendations | 6 |
| **Part 6. Scientific communication and reproducibility** | **10** |
| Report organization and clarity | 4 |
| Reproducible, documented code | 3 |
| Oral presentation (5 minutes) | 3 |
| **Total** | **100** |

# 8 Expectations and Evaluation Philosophy

This project emphasizes **correct methodology, critical thinking, and scientific interpretation**. You will not be graded primarily on the highest accuracy, ROC AUC, or $R^2$.

A simpler model may outperform a more complex model, particularly with limited training samples. Such a result is meaningful when supported by appropriate analysis.

High-quality submissions demonstrate correct separation of development and test data, appropriate preprocessing and cross-validation, careful tuning, accurate performance metrics, thoughtful investigation of overfitting, clear linear/nonlinear comparisons, evidence-supported biological interpretation, and reproducible code.

**Central question:** Not simply "Which algorithm performs best?" but "Why do different algorithms perform differently, and what can we learn from these differences?"